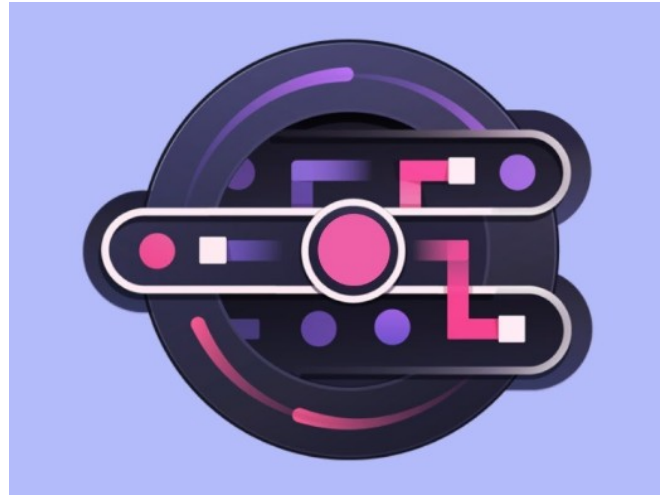




Pipelining et automatisatisation

Déploiement continu et réduction de l'erreur humaine

Pipeline ?

Pipeline : Définition

“Ensemble de tâches récursives automatisées pouvant être séquentielles ou parallèles, permettant d’atteindre le déploiement automatique d’une application sur un environnement donné”

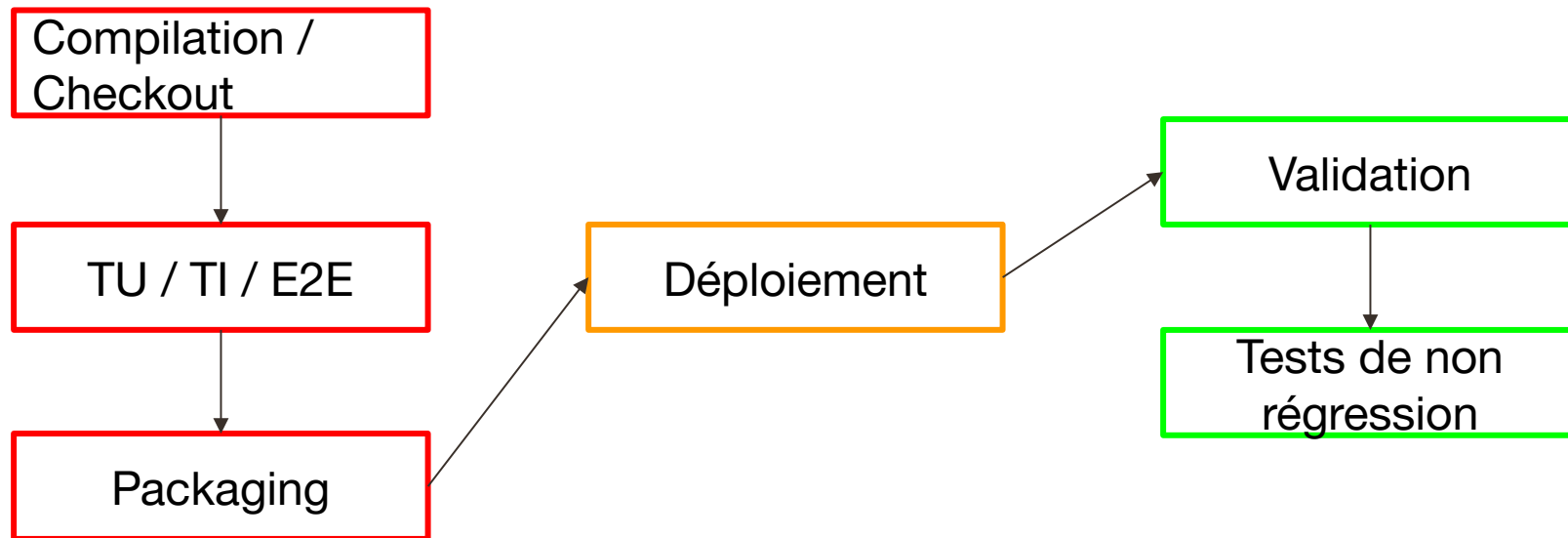
Pipeline : Définition

Un pipeline théorique définit plusieurs étapes :

- Compilation de l'application (si nécessaire)
- Tests (TU, TI, E2E)
- Packaging de l'application
- Déploiement sur les ressources vers lesquelles la distribution a été effectuée
- Validation
- Tests de non régression

Pipeline : Définition

VM (Unix, Windows selon use case)

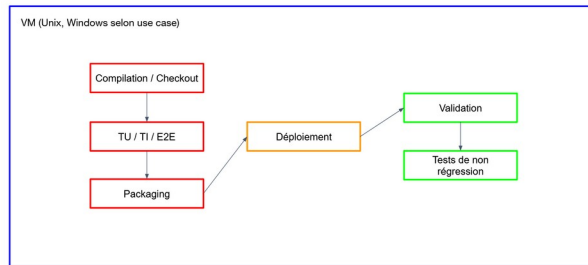


Pipeline : Définition

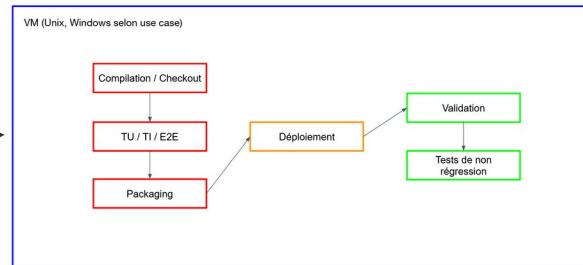
Structure réursive (Meta pipeline)

VM (Unix, Windows selon use case)

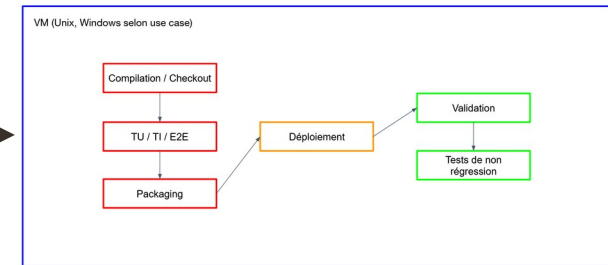
Infrastructure



Back end applicatif



Front end applicatif



Pipeline : Définition

- Peut prendre plusieurs paramètres de configuration
- Peut définir plusieurs variables internes
- Plusieurs niveaux d'exécution
 - Stages
 - Jobs
 - Steps
 - Tasks
- Très dépendant du système utilisé (Dans nos exemples, on considèrera des pipelines Azure puis Github Actions)
- Peut être lancée automatiquement
- Langage déclaratif (YAML)

Pipeline : En pratique

- S'exécute sur une VM approvisionnée par un provider
- Peut exécuter du code de scripting à la volée
- Peut échanger de l'information entre VM / Jobs
- Se lance automatiquement sur une action utilisateur (Hook)

Pipeline : Exemple sur le Cloud Azure



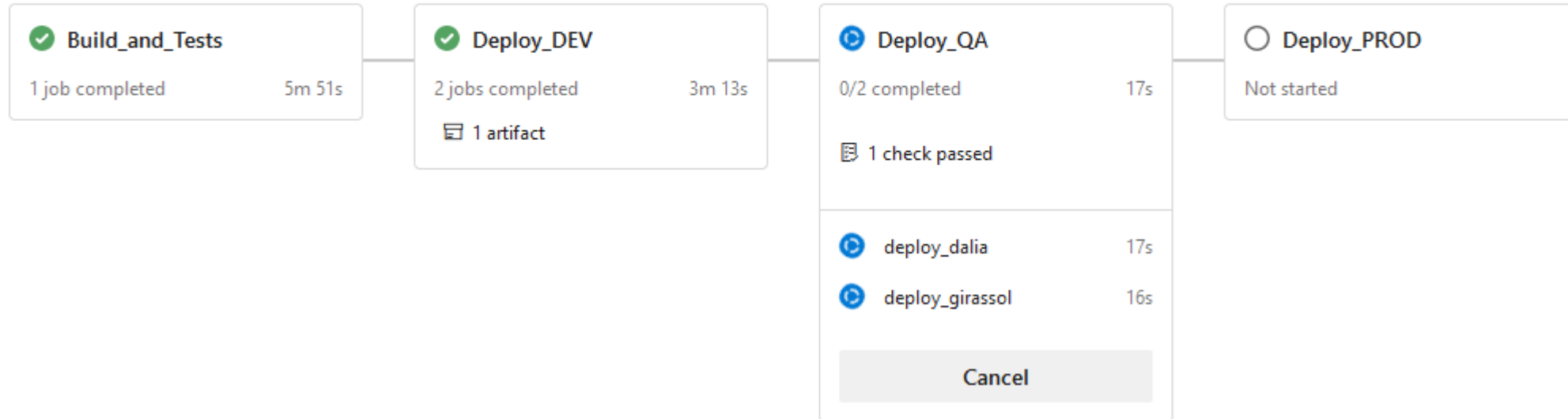
Contexte d'utilisation de la pipeline :

Pouvoir déployer du code Python packagé sur une Application de Fonction (function app)
(Ressource Azure qui s'exécute sur définition de routine / trigger HTTP)

- Ce code doit être configurable pour être déployé sur une function app dédié
- Sur chaque environnement, pour chaque function app, un fichier de configuration doit permettre de déterminer le fonctionnement de chacune d'elle
- Le pipeline doit permettre de déployer sur les 3 environnements de DEV, QA et PROD
- Un passage en QA n'est possible que lorsque le passage en DEV est réalisé, de même pour QA vers PROD
- Les tests devront être lancés en début de déploiement et bloqué le pipeline s'ils échouent

Pipeline : Exemple sur le Cloud Azure

Stages Jobs



Pipeline : Exemple sur le Cloud Azure

Build_and_Tests		
▼	✓ Build_and_Tests	5m 31s
	✓ Initialize job	3s
	✓ Checkout well-monitoring@maste...	1s
	✓ Applicationsecuritytesting	4s
	✓ UsePythonVersion	< 1s
	✓ Authenticate with artifact feed	< 1s
	✓ Install dependencies for pipel...	1m 19s
	⌚ Prepare analysis on SonarQube (...)	< 1s
	✓ Prepare analysis on SonarQube(...)	< 1s
	✓ Run Python unit tests	46s
	✓ Run Code Analysis	3m 9s
	✓ Publish Quality Gate Result	4s
	✓ Post-job: Checkout well-monitori...	< 1s
	✓ Finalize Job	< 1s

Stratégies de flux Git / Déploiement

Release branches :

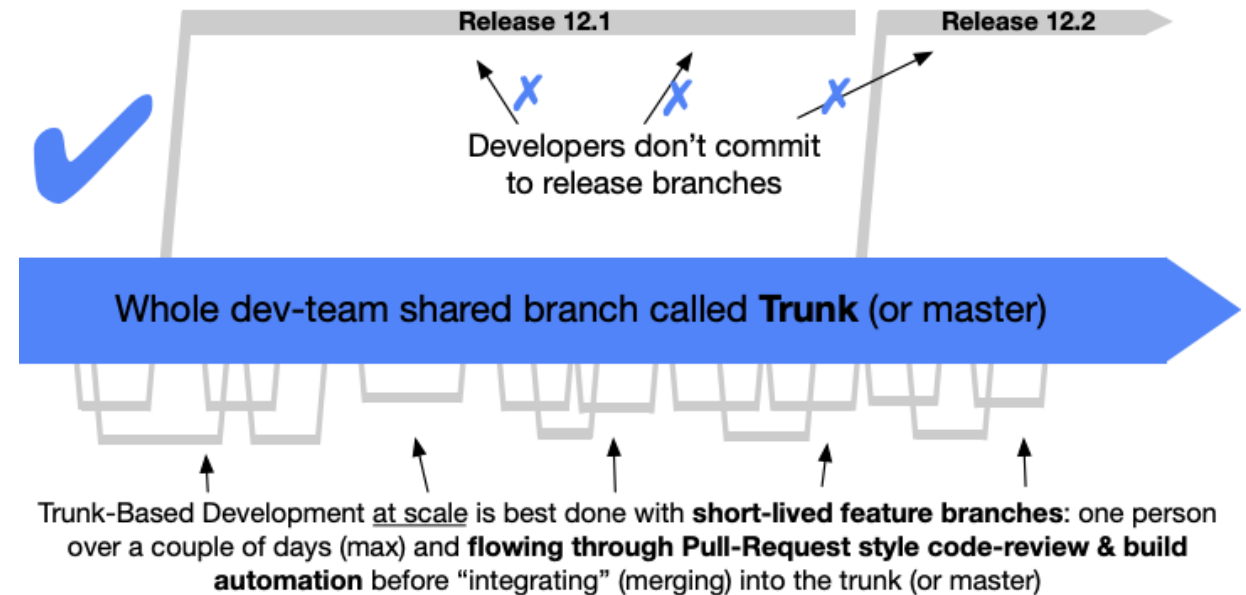
- Source de vérité : master/main/prod



Stratégies de flux Git / Déploiement

Trunk based development :

- Source de vérité : master/main
- Déploiement basé sur une unique branche avec configurations selon environnements



A nous de jouer ?



GitHub Actions

Official documentation : <https://docs.github.com/en/actions>

But : Déployer une application Web générée par Angular CLI vers Github Pages

1. Créer un repo Github
1. Créer une application Angular Hello World
1. Pousser le code sur une branche locale et créer une pull request
1. Créer un pipeline qui va linter le code d'une pull request créée
1. Créer un pipeline de déploiement Github Actions qui s'exécute sur un merge de pull request et va déployer un site vitrine sur Github Pages